



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Michael Darío Gutiérrez Quel Carné: 1199723

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
Una API REST es una interfaz que permite que dos aplicaciones se comuniquen a través de HTTP. Organiza la información en recursos y utiliza métodos como GET, POST, PUT/PATCH y DELETE para consultar o modificar esos recursos.
2. ¿Qué es un endpoint?
Un endpoint es un punto de acceso específico de una API. En la práctica se identifica por la combinación del método HTTP y la URL utilizada para realizar una operación sobre un recurso.
3. ¿Qué diferencia existe entre una URL y un endpoint?
Una URL es la dirección donde se encuentra un recurso o servicio en Internet. Un endpoint es el punto concreto de una API que se utiliza para una operación determinada; por ello, además de la URL, se considera el método HTTP con el que se accede.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
GET se utiliza para consultar información. POST se utiliza para crear un nuevo recurso. PUT reemplaza o actualiza un recurso completo y PATCH modifica parcialmente un recurso. DELETE se utiliza para eliminar un recurso.
5. ¿Qué es JSON y por qué es común en las APIs?
JSON (JavaScript Object Notation) es un formato de texto para representar datos mediante pares clave-valor, objetos y listas. Es común en las APIs porque es ligero, legible y fácil de procesar en diferentes lenguajes de programación.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
Un código de estado HTTP es un número que indica el resultado de una solicitud. 200 significa que la operación fue exitosa; 404 indica que el recurso solicitado no fue encontrado; y 500 indica que ocurrió un error interno en el servidor.
7. ¿Qué información se puede encontrar en la respuesta de una API?
Una respuesta puede incluir el código de estado HTTP, encabezados o headers, y un cuerpo de respuesta. El cuerpo puede contener datos, mensajes, resultados o detalles de error, frecuentemente en formato JSON.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
Postman permite enviar solicitudes y revisar respuestas sin crear primero una aplicación completa. Facilita probar métodos HTTP, URLs, parámetros, headers y cuerpos JSON, además de detectar errores y organizar las pruebas en colecciones.
9. ¿Qué es y para qué sirve Swagger UI?
Swagger UI es una interfaz web que muestra la documentación de una API de forma interactiva. Permite conocer sus endpoints, parámetros, cuerpos de solicitud y posibles respuestas, y también ejecutar pruebas directamente desde el navegador mediante la opción Try it out.
10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?
Postman funciona como cliente para APIs. Se utiliza para construir y enviar solicitudes HTTP, analizar las respuestas, guardar pruebas en colecciones y comprobar que los endpoints funcionan correctamente antes de integrarlos en un sistema o programa.



1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.



- ¿Qué método HTTP utilizó y por qué?
- ¿Qué datos fueron enviados en el body?
- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective



Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.